

Comparison of Machine Learning Algorithms: Cost-Sensitive Evaluation and Multi-class Reduction Strategies

Project Plan

Muhammad Subhan Ahmad

Supervisor: Anand Subramoney

Final Year Individual Project, Department of Computer Science

Royal Holloway, University of London

1 Abstract

Machine learning lets a program learn to solve a class of problems from a training set, rather than being given an explicit solution. Each family of algorithms makes different assumptions about the data. Nearest neighbor methods assume that close points tend to share a label [2], decision trees split the feature space with a sequence of simple rules [1, 2], and support vector machines look for the boundary with the widest margin, optionally through a kernel [1, 3]. When these algorithms are used through a library, the design decisions behind them (the distance metric, the tie-breaking rule, the impurity measure, the optimisation method) are hidden. I want to open that box.

Aim. To implement a range of classifiers from first principles in Python, with no machine learning library, and compare them empirically on real data sets from Kaggle [5, 6].

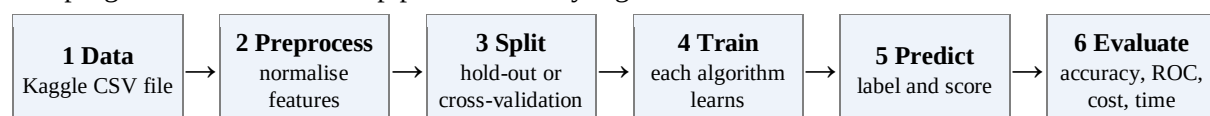
Objectives:

- Write the core algorithms by hand, including k-NN with a k-d tree and several tie-breaking strategies, decision trees with three impurity measures and pruning, kernel nearest neighbours, and a multi-class SVM with a full SMO optimiser [11].
- Check each implementation against Weka and scikit-learn, then compare accuracy, robustness and computational cost using a hold-out test set, k-fold cross-validation and ROC analysis, with normalisation of attributes and examples studied as part of the experiments.
- Extension: study how the algorithms can be adapted to asymmetric costs, as in spam detection, credit risk, fraud and medical diagnosis [7, 8], and whether their ranking changes when judged by expected cost instead of accuracy. A smaller secondary study compares one-against-one, one-against-the-rest and error-correcting codes for multi-class SVMs [9, 10].
- Build the software with an object-oriented design and a modern software engineering process, and write up the theory, implementation issues, process and experiments.

2 What the Project Is and How It Works

In one sentence: a Python program, written from scratch, that learns from labelled examples, tests how well each algorithm predicts labels it has never seen, and compares the algorithms fairly, including when some mistakes cost more than others.

The program follows the same pipeline for every algorithm and data set:



Example. Each row of the Credit Card Fraud data set is one transaction, described by numbers and labelled fraud or genuine. Only about 0.17% of transactions are fraud [5], so a program that always answers “genuine” is right about 99.8% of the time and still useless. Each algorithm therefore learns

from past labelled transactions and, for a new transaction, outputs a label and a score. The project judges the algorithms by ROC curves and by expected cost, where missing a fraud is far worse than a false alarm, instead of by accuracy alone.

Software. A set of Python classes in which every algorithm follows one abstract Classifier interface (train and predict), an experiment runner that reads a configuration and saves tables and plots, and, if time allows, a simple GUI to choose a data set and algorithm and view the results.

3 The Algorithms: How I Will Use Them and What They Give

Algorithm	How I will use it	What it gives
1-NN and k-NN	Store the training examples. For a new example, find the k closest (Euclidean or Manhattan distance, searched with a k-d tree [12]) and let them vote. Ties are broken four ways: random, nearest label, reduce k, distance-weighted [13].	A predicted label and the share of votes for each class, which serves as a score for ROC analysis.
Decision tree	Repeatedly split the data on the feature and threshold that make the groups purest, measured by Gini index, entropy or misclassification error. Stopping rules and pruning limit overfitting.	A label, the class proportions in the leaf (a score), and readable if-then rules.
Kernel nearest neighbours	Same as k-NN, but distances are measured in a kernel feature space (linear, polynomial, RBF), computed directly from the kernel function.	A label and score that can follow curved class boundaries which plain k-NN misses.
Support vector machine	Find the widest-margin boundary that allows some mistakes (soft margin), with a kernel, trained by SMO. For more than two classes, combine binary SVMs by one-against-one, one-against-the-rest or error-correcting codes.	A label and a signed distance from the boundary (a score), and the support vectors that define it.
Cost-sensitive versions	Use a cost matrix to shift the decision threshold, weight training examples, or change the algorithm's rule (weighted votes, cost-weighted impurity, class-specific C).	Predictions that minimise expected cost, not only the number of errors.
Evaluation tools	Hold-out test set, stratified k-fold cross-validation, confusion matrix, ROC curve and AUC, cost curves, and paired significance tests.	Accuracy, AUC, expected cost, timings, and whether differences between algorithms are real.

Everything is written from scratch. NumPy is used only for array arithmetic and Matplotlib only for plots. The complex parts I will build myself are:

- neighbour search with a k-d tree, checked against brute-force search;
- decision tree induction with exhaustive threshold search, three impurity measures and pruning;
- kernel functions and distances in kernel feature space;
- a full SMO optimiser [11] with a kernel cache, working-set heuristics, soft margin and class-specific C;
- the multi-class reductions (one-against-one, one-against-the-rest, error-correcting codes);
- stratified cross-validation, ROC and AUC from a threshold sweep, cost curves, normalisation and significance tests.

Weka and scikit-learn are used only afterwards, to check that my results are correct.

4 Data Sets

Data will be downloaded from Kaggle (free account) and kept outside the repository, on university online storage. Each data set's page and licence will be cited. Where a Kaggle copy of a UCI data set is unavailable, I will use the original UCI file and cite it.

Data set	Source	Used for
Credit Card Fraud Detection (284,807 transactions)	Kaggle, ULB [5]	Extension: extreme imbalance and cost-sensitive learning. Subsamples for development, the full set for scaling tests.
Breast Cancer Wisconsin (Diagnostic)	Kaggle, UCI ML [6]	Core comparison and medical diagnosis, where missing a malignant case is worse.
Spam e-mail (Spambase)	Kaggle or UCI [4]	Spam detection, where blocking a genuine e-mail is costly.
Credit risk (German Credit)	Kaggle or UCI [4]	Credit risk with an unequal cost matrix.
Iris and Wine	Kaggle or UCI [4]	Small sanity checks and normalisation (Wine).
A multi-class image data set, for example a subsample of Digit Recognizer	Kaggle	Secondary study of multi-class reduction (10 classes).
Artificial data (blobs, XOR, circles)	Generated by my program	Early testing, where the right answer is known.

5 Choice of Programming Language

I compared three candidates and, with my supervisor's agreement, chose Python.

Language	Strengths for this project	Weaknesses for this project
Python	Widely used in machine learning practice. NumPy array operations run in compiled code, Matplotlib gives ROC and cost curves with little code, and pytest and abstract base classes support test-driven, object-oriented design.	Plain loops are slow, and dynamic typing lets some errors survive until run time.
Java	Static typing, mature JUnit testing, and Swing or JavaFX for a GUI.	More boilerplate and little built-in numerical or plotting support, so more time goes on infrastructure.
C++	The fastest of the three.	Manual memory management and a heavier build and test set-up make it the slowest to develop in.

Python lets me spend my time on the algorithms and experiments. Its speed weakness is handled by vectorising with NumPy, the k-d tree, profiling early, and using subsamples while developing. All algorithms are timed in the same language, so comparing their running times stays fair.

6 Timeline

I will show progress at the weekly supervisor meetings. Week numbers will be matched to the real term dates once they are confirmed.

6.1 Term 1: early programs and reports

Week	Work and milestones
Week 1	Background reading (Mitchell [2]; Hastie et al. [1]). Set up the GitLab repository, Python environment and Overleaf report. Create the Kaggle account and download the data sets. Sketch the abstract Classifier class.
Week 2	Program: 1-NN and k-NN from scratch with Euclidean and Manhattan distance and the four tie-breaking strategies. Generate artificial data (blobs, XOR, circles).
Week 3 to 4	Program: decision trees with threshold search, Gini, entropy and misclassification error, stopping rules and pruning. pytest tests for k-NN and trees on the artificial data.
Week 5	Milestone: both early reports, on k-NN with tie-breaking and on decision trees with different measures of uniformity.
Week 6 to 7	Evaluation framework: hold-out split, stratified cross-validation, confusion matrix, ROC and AUC, normalisation. Load and clean the Kaggle data sets.
Week 8	First experiments for RQ1 to RQ3, checked against Weka and scikit-learn.
Week 9	Programs: kernel k-NN, and the k-d tree speed-up for k-NN, checked against brute force.
Week 10 to 11	Milestone: interim report and presentation.

6.2 Term 2: advanced algorithms, extension and final report

Week	Work and milestones
Week 1 to 3	Finish kernel k-NN. Implement the SVM with a full SMO optimiser, then one-against-one multi-class.
Week 4	Full comparison of all algorithms on accuracy, training time and prediction time, including scaling on the larger Kaggle data.
Week 5 to 6	Extension: cost-sensitive k-NN, trees and SVM on the fraud, spam, credit and breast cancer data, with expected cost and cost curves.
Week 7	Secondary study: one-against-the-rest, one-against-one and error-correcting codes on the multi-class data set.
Week 8	Simple GUI with plots of ROC curves and decision boundaries, if time allows.
Week 9	Final experiments, significance tests and analysis.
Week 10 to 11	Milestone: final report and viva preparation, with buffer time.

7 Risks and Mitigations

Risk	What I will do
Hardware failure	Commit regularly to the university GitLab, keep the report on Overleaf with a copy in the repository, and keep data sets on university online storage.
Poor estimation of tasks	Break work into small tasks, check progress against the timeline every week, and keep buffer weeks at the end of each term.
Report and code out of balance	Write each report section as soon as the matching program works, instead of leaving the writing to the end.
Incorrect programs or flawed experiments	Test each algorithm on artificial data with known answers, cross-check with Weka and scikit-learn, fit preprocessing inside each cross-validation fold, and never tune on the test set.

Risk	What I will do
The from-scratch code is complex (SMO, k-d tree)	Build in stages, simplified version first. Check the k-d tree against brute force and the SVM against Weka. Start the SVM in the first week of Term 2, working from Platt's paper [11].
Scope creep	The core algorithms and their evaluation come first. The secondary study and the GUI are the first things I drop if time runs short.
Python is too slow on large data	Vectorise with NumPy, use the k-d tree, profile early, and use subsamples of the large Kaggle data until the pipeline is correct.
Kaggle data problems	Kaggle needs an account, files can be large (the fraud data is about 150 MB) and are not committed to Git, and some data sets may not be available. I will cite each licence and fall back on the original UCI files if needed.
Learning curve	I am still building my programming skills, so I will work in small steps, test each piece, review all code (including any AI-generated code) until I can explain it, declare AI use in AI.md, and ask my supervisor early if I am stuck.

8 Bibliography

- [1] T. Hastie, R. Tibshirani and J. Friedman. The Elements of Statistical Learning, 2nd ed. Springer, New York, 2009.
- [2] T. M. Mitchell. Machine Learning. McGraw-Hill, New York, 1997.
- [3] V. N. Vapnik. The Nature of Statistical Learning Theory, 2nd ed. Springer, New York, 2000.
- [4] UCI Machine Learning Repository. <https://archive.ics.uci.edu>
- [5] Machine Learning Group, ULB. Credit Card Fraud Detection. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [6] UCI Machine Learning. Breast Cancer Wisconsin (Diagnostic) Data Set. Kaggle. <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
- [7] T. Fawcett. An introduction to ROC analysis. Pattern Recognition Letters 27(8), 2006.
- [8] C. Elkan. The foundations of cost-sensitive learning. IJCAI, 2001.
- [9] C.-W. Hsu and C.-J. Lin. A comparison of methods for multiclass support vector machines. IEEE Trans. Neural Networks 13(2), 2002.
- [10] T. G. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. JAIR 2, 1995.
- [11] J. Platt. Sequential minimal optimization: a fast algorithm for training support vector machines. Microsoft Research Tech. Report, 1998.
- [12] J. L. Bentley. Multidimensional binary search trees used for associative searching. Communications of the ACM 18(9), 509–517, 1975.
- [13] S. A. Dudani. The distance-weighted k-nearest-neighbor rule. IEEE Transactions on Systems, Man, and Cybernetics SMC-6, 325–327, 1976.